IA para Desarrolladores

Sesión 4 · Hacer — del plan al primer componente

CURSO · 20 HORAS · 10 SESIONES

Agenda de hoy

5'	25'	10'	20'	20'	30'	10'
Repaso: la especificación	Su repositorio: reglas, banco y aprobar el spec.md	La estructura por capas	Demo: el Arquitecto escribe el plan	Demo: el Desarrollador, prueba primero	Práctica: mi plan y el primer paso	Preguntas, tarea y cierre

Hito del curso: el primer componente de su reto con las pruebas en verde.

Dónde vamos en el ciclo

1 · Intención

`intent.md`

Sesión 2 ✓

→

2 · Especificación

`spec.md`

Sesión 3 ✓

→

3 · Plan

`plan.md`

Hoy

→

4 · Construcción

`código y pruebas`

Hoy empieza

→

5 · Revisión

`revision.md`

Sesión 5

→

6 · Entrega

`CAMB-####`

Sesión 8

Lo que hoy se suma al agente del equipo (versión 2)

Dos agentes por rol

Arquitecto (planea, no toca código) y Desarrollador (construye, prueba primero).

Plan y nombres

La skill /plan-tecnico y validar_nombre, la herramienta del banco que revisa cada nombre.

Capas que se verifican

crear_estructura.py arma el proyecto; verificar_capas.py revisa la regla.

Repaso: la especificación de la sesión 3

- ¿Tiene requisitos (RF) con criterios Dado / Cuando / Entonces y valores concretos?
- ¿Tiene reglas de negocio (RN), supuestos y preguntas abiertas?
- ¿Dice Estado: borrador y tiene su commit?
- ¿Qué cosas tuvo que suponer porque no las sabía?

Todavía no se aprueba

Se escribió sin las reglas ni el servidor del banco. Primero ponemos al día el repositorio, la revisamos con el banco y después la aprobamos.

Su repositorio, completo

Las reglas, el servidor del banco y, al final, aprobar el spec.md

Primero: ponga al día su repositorio

Desde la carpeta del curso, con el material nuevo (zip o git pull):

```
bash proyectos/poner_al_dia.sh ../mi-reto
```

1

Guarda su trabajo

Hace commit de lo suelto en docs/sdlc. Sus intent.md y spec.md nunca se tocan.

2

Trae lo que falta

Reglas, skills, servidor MCP y los agentes de hoy. No copie carpetas a mano.

3

Respeta lo suyo

Si cambió un archivo, la versión nueva queda al lado como .nuevo.

4

Verifica

verificar_equipo.py en OK. Luego /mcp en claude o Start en .vscode/mcp.json.

Cada sesión empieza igual: material nuevo del curso y poner_al_dia.sh. Si se saltó una sesión, el mismo comando lo pone al día.

Qué lee el agente, y cuándo

Pieza	Dónde está	Cuándo la usa el agente	Ejemplo
Instrucciones	CLAUDE.md · AGENTS.md .github/copilot-instructions.md	Siempre, al abrir la sesión	Python, datos sintéticos, en español
Skill	.claude/skills/ · .github/skills/	Cuando usted la llama	/spec, /plan-tecnico
Regla	.claude/rules/ · .github/instructions/	Sola, al tocar un archivo que coincide	Escribe en tests/ → regla de pruebas
Herramienta del banco (MCP)	.mcp.json · .vscode/mcp.json	Cuando necesita un dato del banco	glosario("usura")
Agente	.claude/agents/ · .github/agents/	Cuando usted lo abre (hoy)	claude --agent arquitecto

Todo vive en el repositorio: cualquiera que lo clone tiene el mismo agente. Las skills y reglas de Copilot (.github) se generan desde .claude con sincronizar_skills.py.

¿Qué es una regla?

Una instrucción para el agente que solo aplica a ciertos archivos: las pruebas, las APIs, los artefactos del ciclo.

No se llama: se carga sola cuando el agente va a leer o escribir un archivo que coincide con sus paths.

Mantiene corto AGENTS.md: lo de siempre va ahí; lo de cierto tipo de archivo, en una regla.

Nace de un error repetido: si el agente hizo algo distinto a como lo hace el equipo dos veces, se vuelve regla.

```
---
description: Pruebas del equipo
paths:
  - "tests/**/*.py"
---
# Reglas para las pruebas
- Funciones test_<regla>_<caso>
- Docstring con su clave de banco.yaml
- Bordes con parametrize
```

`description`
para qué es

`paths`
a qué archivos aplica

`cuerpo`
lo que el agente debe hacer

Claude Code: .claude/rules/*.md con paths. Copilot: .github/instructions/*.instructions.md con applyTo. Se escribe una vez y sincronizar_skills.py genera la de Copilot.

Así se carga una regla: anoche, en demo-equipo

```
❯ /spec simulador-credito-consumo

  Searched for 1 pattern, read 3 files, called banco 3 times
   └  Loaded .claude\rules\artefactos.md
```

1

Nadie la pidió

Iba a escribir en docs/sdlc/ y Claude Code cargó artefactos.md por sus paths.

2

Cambió el resultado

La cuota fija no está en banco.yaml: la pasó de regla a supuesto S-01.

3

Dejó el borrador

Estado: borrador. Solo el dueño del producto aprueba.

Para verlo usted: la línea Loaded … aparece en la terminal; Ctrl+O la expande. En Copilot, la regla aparece entre las referencias de la respuesta.

Las reglas que vamos a usar

Regla	Se carga con	Qué pide	Cuándo la verá
artefactos	docs/sdlc/**/*.md	Dejar borrador; citar la clave de banco.yaml; lo que no está, a Supuestos	Desde la sesión 3 en cada intent, spec y plan; hoy en plan.md
pruebas	tests/**, test_*.py	Nombres en español, docstring con la regla, bordes con parametrize	Hoy: el Desarrollador escribe la prueba primero
capas	src/**/*.py	Cada capa importa solo las permitidas; dominio sin frameworks	Hoy: el código del primer componente
api	**/api/**, adaptadores/api*	Rutas /v1/<plural>, Pydantic en el adaptador, montos como texto	Cuando su reto exponga una API
etl	**/etl/**, *_etl.py	extraer, transformar y cargar puras; utf-8; umbrales de banco.yaml	En los retos con procesos ETL
la suya	la que usted elija	Algo que su asistente hizo distinto al equipo dos veces	Tarea: copie una y cámbiela

Una regla aconseja; un verificador o un hook obliga. Hoy capas ya tiene su verificador (verificar_capas.py); en la sesión 6 los hooks lo corren solos.

El servidor del banco, en su repositorio



Claude Code · .mcp.json

```
"banco": { "type": "stdio",
  "command": "<su Python>",
  "args": [".../mcp_banco/servidor.py"] }
```

Copilot · .vscode/mcp.json

```
"banco": { "type": "stdio",
  "command": "<su Python>",
  "args": ["${workspaceFolder}/.../servidor.py"] }
```

En la sesión 3 lo probamos en una carpeta aparte. Ahora es parte de mi-reto: configurar_mcp.py escribe la ruta de su Python en los dos archivos. Cambia banco.yaml y la respuesta cambia, sin reiniciar.

Demo: comprobarlo en demo-equipo

1 · ¿Está conectado?

```
/mcp
```

Debe aparecer banco, conectado. En Copilot: Start en .vscode/mcp.json.

2 · ¿Consulta al banco?

```
Según el glosario del banco,
¿qué es usura y dónde está
su valor?
```

Llama a glosario: tasa_maxima_ea, de banco.yaml. No lo adivina.

3 · ¿Qué reglas aplican?

```
¿Qué reglas de .claude/rules
se aplican a
tests/test_cuota.py
y a
src/.../dominio/cuota.py?
```

pruebas para el primero; capas para el segundo. Lo dicen sus paths.

Si algo falla: verificar_equipo.py dice qué falta. Si /mcp no muestra banco, salga y vuelva a abrir claude en mi-reto.

Ahora sí: revisar y aprobar el spec.md

1 · Revisar con el banco

```
Revisa
docs/sdlc/<caso>/spec.md
con las herramientas del
banco.
Lista las reglas sin clave o
política y los términos fuera
del glosario. No cambies el
archivo.
```

El agente llama a banco y carga la regla artefactos.

2 · Corregir

```
Aplica las correcciones de la
lista. No cambies el Estado.
```

O regenere: /spec <caso>. Lo que no está en banco.yaml va a Supuestos.

3 · Aprobar (usted)

```
Estado: aprobado
Aprobado por: <su nombre>
  (dueño del producto)

git commit -am "Spec aprobada"
```

La persona aprueba; el agente nunca. El commit es la firma.

Así quedó la del simulador: cita politicas.datos, la cuota fija pasó a supuesto y aparecieron svc-tasas y svc-clientes como preguntas abiertas.

La estructura por capas

Dónde va cada cosa, y quién puede importar a quién

Cinco capas, una dirección



las dependencias apuntan hacia el dominio

El dominio no conoce a nadie: ni Streamlit, ni la red, ni banco.yaml. Por eso se prueba en milisegundos.

Lo de afuera entra por un puerto: el caso de uso no sabe si la tasa viene de memoria o de svc-tasas.

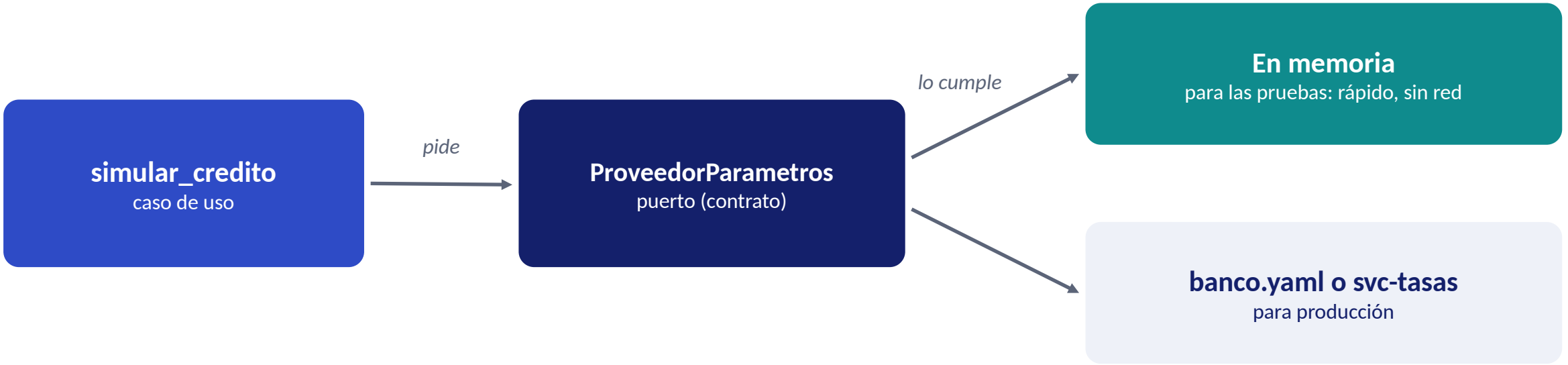
Se crea con un comando: python herramientas/crear_estructura.py <paquete>

Quién puede importar a quién

Capa	Qué va aquí	Puede importar	En el simulador
dominio	Entidades, cálculos, validaciones	Nada del proyecto; ningún framework	calculo_cuota.py, capacidad_pago.py
puertos	Contratos (typing.Protocol)	dominio	proveedor_parametros.py
casos_uso	Orquesta dominio y puertos	dominio, puertos	simular_credito.py
adaptadores	Implementan los puertos	dominio, puertos, casos_uso	proveedor_parametros_en_memoria.py
app	La entrada	Todas	interfaz_streamlit.py

La regla vive en .claude/rules/capas.md y se carga cuando el agente toca src/. Pero una regla aconseja: falta algo que la verifique.

Puertos y adaptadores: lo de afuera entra por una puerta



Por eso el catálogo importa: cada servicio de catalogo_servicios entra por un puerto, y el agente lo deja escrito en el plan (sección 4).

La regla se verifica, no se pide

```
$ python herramientas/verificar_capas.py
Regla de capas: hay violaciones
  ✗ simulador/dominio/calculo.py:1  el dominio no
      puede importar el framework streamlit
  ✗ simulador/dominio/calculo.py:2  dominio no puede
      importar adaptadores (simulador.adaptadores)
```

Lee las importaciones de cada archivo de src/ y las compara con la tabla de capas.

Sale con error si algo no cumple: sirve para el agente, para las pruebas y para CI.

El Desarrollador lo corre al final de cada paso. En la sesión 6 lo corre un hook, sin pedirlo.

Una skill aconseja, un hook obliga: hoy la regla ya se puede comprobar; en la sesión 6 se vuelve obligatoria.

Dos agentes, dos permisos

Quien planea no construye; quien construye sigue el plan

Arquitecto y Desarrollador

Arquitecto

Planea el cómo

Lee la especificación aprobada y el código que exista

Consulta catálogo, políticas y validar_nombre

Escribe solo docs/sdlc/<caso>/plan.md con /plan-tecnico

Sin terminal: no ejecuta comandos ni toca src/

Desarrollador

Construye el plan aprobado

Lee el plan aprobado; si no lo está, se detiene

Prueba primero en cada paso: rojo, implementar, verde

Corre pytest y verificar_capas.py; marca el paso en el plan

Si duda, pregunta: no improvisa otro diseño

Entre los dos, una persona: el líder técnico aprueba el plan. El agente nunca aprueba su propio trabajo.

El mismo agente en las dos herramientas

Claude Code · .claude/agents/arquitecto.md

```
---
name: arquitecto
description: Diseña el plan técnico...
tools: Read, Glob, Grep, Write, Edit,
  Skill, mcp__banco__catalogo_servicios,
  mcp__banco__validar_nombre, ...
model: inherit
---
Usted es el Arquitecto...
- Solo escribe docs/sdlc/<caso>/plan.md
```

Copilot · .github/agents/arquitecto.agent.md

```
---
name: Arquitecto
description: Diseña el plan técnico...
tools: ['read', 'search', 'edit', 'banco/*']
handoffs:
  - label: Implementar el plan aprobado
    agent: Desarrollador
    prompt: El líder técnico aprobó el plan...
---
Usted es el Arquitecto...
```

tools decide qué puede hacer: el Arquitecto no tiene terminal (Bash / execute).

Claude Code: claude --agent arquitecto. **Copilot:** selector de agentes; handoffs pone el botón para pasar al Desarrollador.

validar_nombre: la cuarta herramienta del banco

✓ caso_uso **`simular_credito`**
OK: cumple las convenciones

✗ caso_uso **`calculate_loan`**
Empieza con verbo en infinitivo; palabras en inglés

✓ clase **`SimulacionCredito`**
OK: PascalCase en español

✗ modulo **`calcularCuota`**
Debe ir en snake_case

✓ ruta **`/v1/creditos/{id_credito}`**
OK: versión y plural

✗ servicio **`creditos-api`**
Se llama svc-<dominio>

Las reglas de nombres salen de banco.yaml: convenciones y politicas.nombres, incluida la lista de palabras en inglés que no se usan.

Qué hace /plan-tecnico

1

¿Aprobada?

Lee spec.md. Si no dice aprobado, se detiene.

2

Consulta

Catálogo, políticas de nombres y pruebas, y validar_nombre para cada nombre.

3

Diseña en capas

Qué archivo en cada capa, qué RF y RN cubre, qué entra por puertos.

4

Prueba por criterio

Cada criterio de aceptación se vuelve una prueba con su nombre.

5

Pasos en orden

Prueba primero, y cuál es el primer componente. Queda en borrador.

Se llama /plan-tecnico y no /plan porque /plan ya es el modo plan de las herramientas.

Así se ven los pasos de un plan.md

```
## 7. Pasos (en orden, prueba primero)
- [ ] Paso 1 · Estructura. crear_estructura.py simulador_credito
- [ ] Paso 2 · Parámetros y errores del dominio.
- [ ] Paso 3 · Validación (RF-01). Prueba: las 16 de
      test_validacion_condiciones.py. Verlas fallar.
- [ ] Paso 4 · Cuota (RF-02). Listo cuando: las tres
      cuotas de referencia coinciden al peso.
- [ ] Paso 5 · Total y costo (RF-03).
...
Primer componente: pasos 1 a 5.
```

Cada paso dice qué prueba, qué archivo y cuándo está listo.

Las casillas las marca el Desarrollador: el plan muestra el avance.

Los valores de referencia salen del spec.md, calculados aparte.

¿Cuándo se aprueba un plan.md?

- Cada componente está en la capa correcta y dice qué RF y RN cubre
- Lo de afuera entra por un puerto con adaptador en memoria
- Cada criterio de aceptación tiene su prueba
- Los nombres están validados
- Pasos en orden, prueba primero, con el primer componente señalado
- Solo dependencias aprobadas; nada fuera del alcance

Lo aprueba el líder técnico

```
Estado: aprobado
Aprobado por: <nombre>
  (líder técnico)

git add docs/sdlc
git commit -m
  "Plan técnico aprobado"
```

Prueba primero

El Desarrollador construye el primer componente

Cada paso, el mismo ciclo



¿Por qué verla fallar? Una prueba que nunca falló puede no estar probando nada. Si pasa antes de implementar, algo está mal.

Lo que construyó en el ensayo

```
def test_cuota_referencia_doce_meses():
    """Regla: convenciones.dinero y glosario.cuota_fija
    (referencia 934.525)."""
    cuota = calcular_cuota(Decimal("10000000"), 12,
                           Decimal("0.24"))
    assert cuota == Decimal("934525")
```

38

pruebas en verde

4

pasos del plan marcados

OK

regla de capas (10 archivos)

0

float en src/

Y se detuvo a preguntar: la regla de pruebas pedía agrupar los bordes y el plan los listaba uno por uno. Siguió el plan y lo dejó para decisión del líder técnico.

Manos a la obra: mi plan y el primer paso

8'

Al día y spec aprobado

poner_al_dia.sh, revisar su spec.md con el banco y aprobarlo (lámina 13).

12'

Plan con el Arquitecto

claude --agent arquitecto
o el selector de Copilot.
/plan-tecnico <caso>

5'

Revisar y aprobar

Con la lista. Estado: aprobado, su nombre como líder técnico, y commit.

5'

Paso 1 con el Desarrollador

claude --agent desarrollador
Implementa el paso 1 del plan aprobado de <caso>.

¿Algo no funciona? verificar_equipo.py dice qué falta. Si el agente no existe, abra la terminal en mi-reto.

Para casa: el primer componente

1

Terminarlo

Implementa los pasos 2 a <n> del plan aprobado: de a dos o tres pasos por conversación.

2

Revisar cada paso

¿La prueba verifica el criterio? ¿El código está en la capa correcta?

3

Comprobar

python -m pytest -q en verde y verificar_capas.py en OK. Commit.

4

Entregar

plan.md y la salida de pytest y verificar_capas por el canal, antes de la sesión 5.

Hito: primer componente con pruebas en verde. Pendiente de la sesión 3: su regla propia (TAREA.md). Lectura: Claude Code 101 y GH-600.

Para la sesión 5

1

GitHub

El trabajo pasa a un repositorio compartido: ramas y pull requests.

2

Dos agentes más

El Depurador encuentra un error con una prueba que falla; el Revisor revisa sin poder editar.

3

Lo que traen

Su primer componente en verde y su plan.md aprobado.

Quien planea no construye; quien construye sigue el plan; una persona aprueba entre los dos.